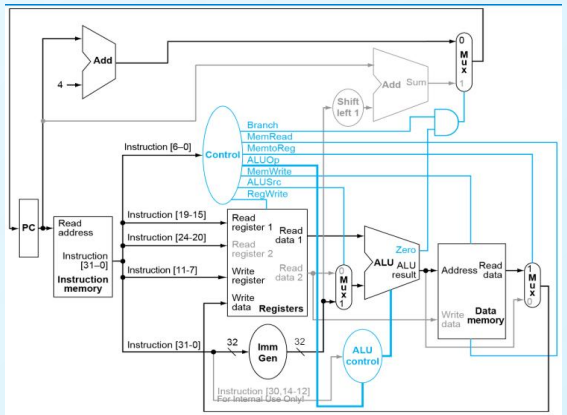
Computer Organization

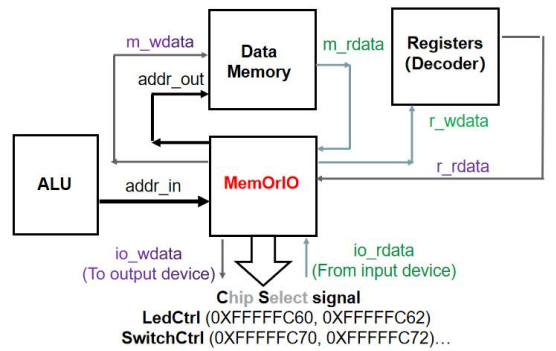
Lab12 SOC

'single' cycle CPU,
clock divider, I/O(MMIO)

Topic

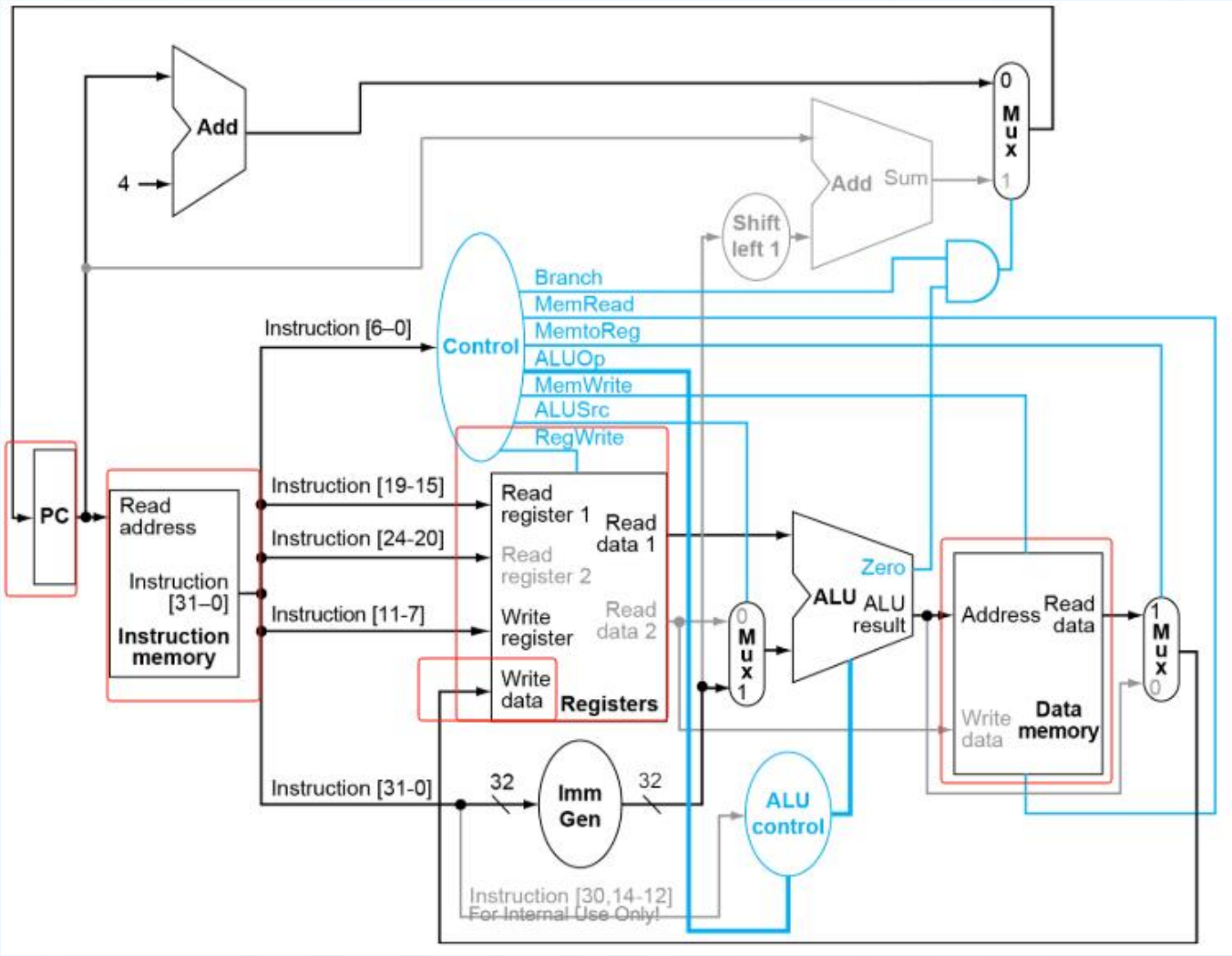
- **SOC**
  - A ‘single’ cycle CPU
  - Clock Divider(verilog)
  - Input/Output device





- **CPU work with I/O device**
  - MMIO(Memory-Mapped Input Output)
  - Modification on CPU
    - CPU TOP(add new input and output port)
    - MemOrIO
    - Controller +

A ‘single’ cycle CPU



Q1. Does it **take time** for signals to be processed and transmitted within the module, as well as between modules?

Q2. Which sub modules within CPU **need the trigger from the clock**? When does the following event occur in a clock cycle?

1-1) **IFetch**: **update** the value of PC register
1-2) **IFetch**: **fetch** the instruction according to the value of PC

2-1-1) **Controller**: generate the **control signals**
2-2-1) **Decoder**: **get** the value of register(s)
2-2-2) **Decoder**: **generate** the extended **immediate**

3-1) **ALU**: **get** the **operands**
3-2) **ALU**: **generate** the **calculation result**

4-1) **Dmemory**: **get** the **address**(from ALU) and **data**(from Decoder)
4-2) **Dmemory**: **read out the data**

5-1) **Decoder**: **write back the data**

Clock(1)

The clock on the EGO1 development board is **100Mhz.**

Q: Is the **100Mhz** (cycle 10ns) clock work ok for the 'single cycle' CPU?

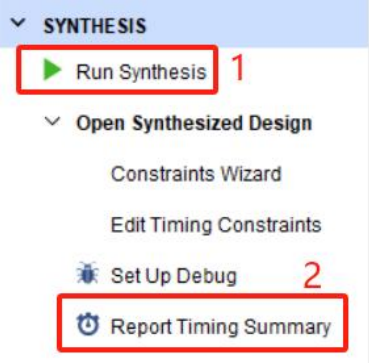
A: Using vivado to check

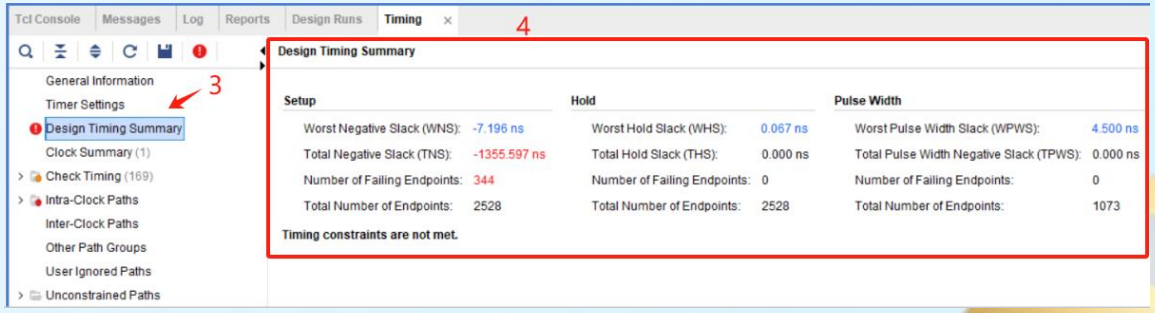
- **step1**: Creat a constraints file with cpu_top, add a constraint description item about the clock(line 5)

```
1  set_propery IOSTANDARD LVCOMS33 [get_ports clk_top]
2  set_propery IOSTANDARD LVCOMS33 [get_ports rst_top]
3  set_propery PACKAGE_PIN P17 [get_ports clk_top]
4  set_propery PACKAGE_PIN P15 [get_ports rst_top]
5  create_clock -period 10.00 -name clk [get_ports clk_top]
```

- **step2**: Run synthesis, Check the "Report Time Summay".

If the TNS or WNS have negative value, it means that the design may not run properly, and circuit design optimization is needed to reduce logic delay or lower clock frequency to solve this problem.



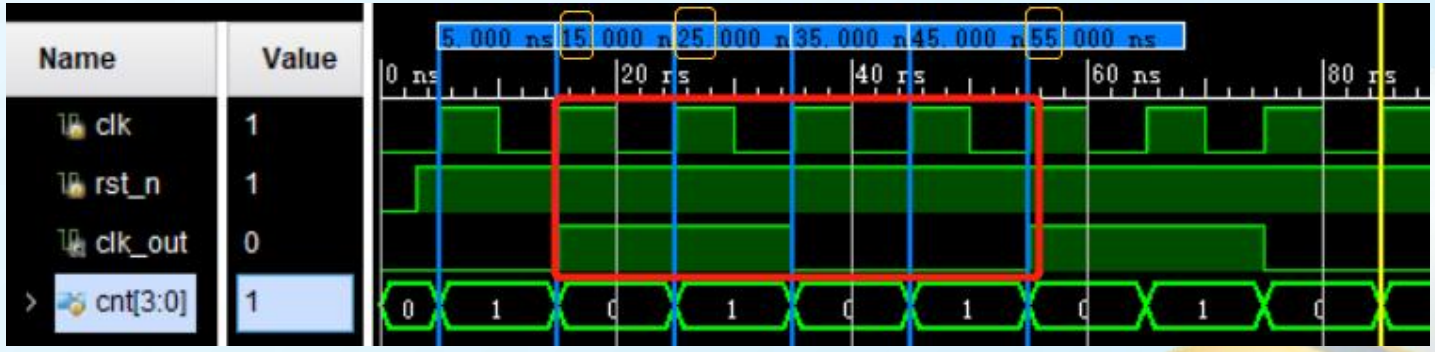


Clock(2)

The frequency divider can be used for lower clock frequency, instantiate BUFG to convert the original frequency-divided signal into a high-quality global clock, and use clk_out_global as the CPU clock.

```
`timescale 1ns / 1ps
//////////////////////////////////////////////////////////////////////////////////
module clk_div(input clk,rst_n,output reg clk_out);
    parameter  period = 4;
    reg [3:0] cnt;
    always@(posedge clk,negedge rst_n)
    begin
        if(~rst_n)begin
            cnt <=0;
            clk_out<=0;
            end
        else
            if(cnt==((period>>1) - 1)) begin
                clk_out <= ~clk_out;
                cnt <=0;
            end
            else begin
                cnt <= cnt+1;
            end
    end
endmodule
```

```
//clk_out here is  Logic-generated high-skew clock
//clk_out_global here is Low-skew output from global clock tree

BUFG BUFG_inst ( .O(clk_out_global), .I(clk_out)  );
```



Test a ‘single cycle’ CPU

Q. How to test the CPU ?

Q1.how to determine the testcase and make it work on CPU?
A1. asm file ->coe files->Instruction Memory & Data Memeory in CPU

Q2. how to check the testing result?
A2-1. Check the internal signal in the CPU
1)Do the simulation (learnt in lab10)

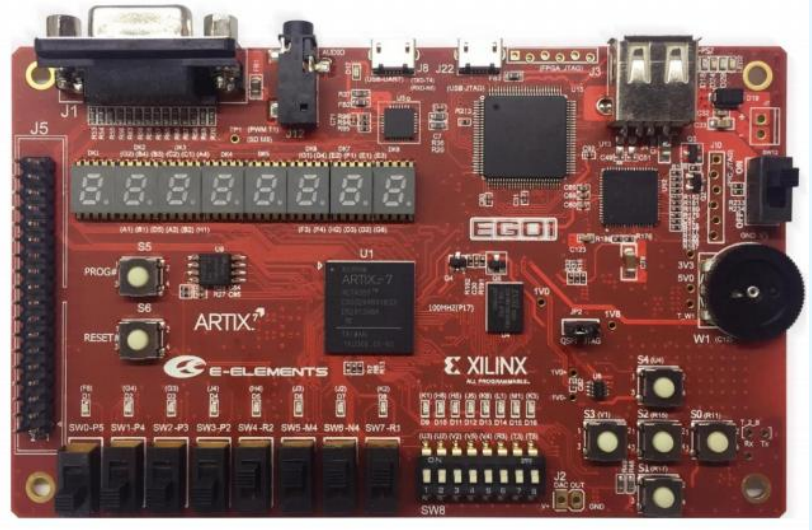
A2-2. Make CPU work with I/O device, check the external signal of CPU
- ✓ **1) Do the simulation**
- ✓ **2) Do the testing on develop board(EGO1/Minisys) with FPGA chip embeded**
  - ✓ Generate bitstream file of CPU -> ‘program device’ to the FPGA embed on EGO1/Minisys -> test on develop board(EGO1/Minisys) with FPGA chip embeded

I/O Interface & MMIO with CPU

The handbook of board **EGO1** could be found in the directory "**labs\Tools\EGO1_user_manual**" on the course **BlackBoard sit.**

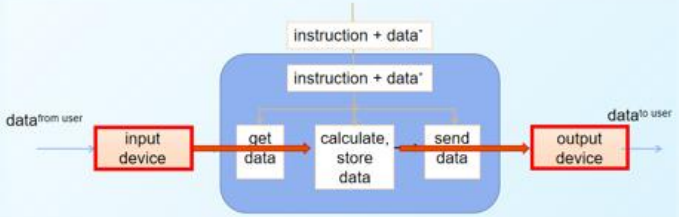
Ego1 User Manual

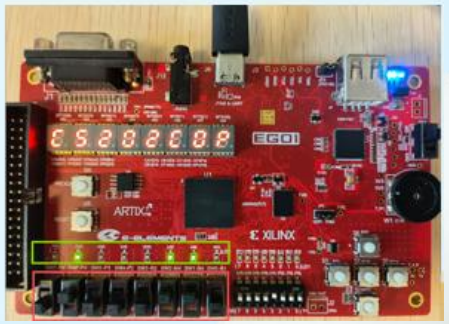
Attached Files: EES328_Test_Flow_V2.1_20171211-EGO1.rar (19.014 MB)
EGo1.xdc (16.503 KB)
Ego1_UserManual_v2.2.pdf (803.057 KB)



We have practiced a Cropped CPU on EGO1 in lab9, **MMIO(Memory-Mapped Input Output)** is used.

```
#assmbly source file  in RISC-V
.data              # data* is 4bytes, its initial value is 0
      buf: .word 0x0000

.text              # instructions
main:
      # in  'tailored General purpose processor'
      # the value in register x31 is 0xFFFF0000.  ONLY register x1 is writable

      lw   x1, 0(x31)      #copy data from 0xFFFF0000(switches) to register x1
      sw   x1, 8(x31)      #copy data from register x1 to 0xFFFF0008(leds)

      j main               # jump to the instructions labled by main
```





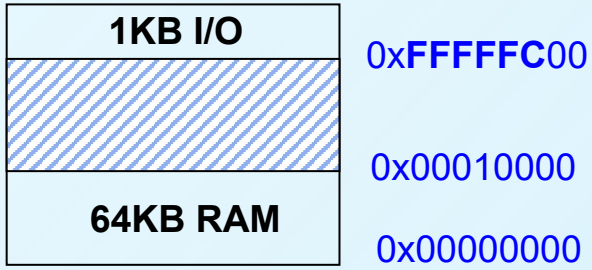
A Simple Design on the I/O Interface

This part mainly accomplishes the following work:

1. Add I/O function
2. 16-bit LED design
3. 16-bit DIP Switch design

This is only one of the design solutions for I/O related data bus. **Please develop a solution that suits your design needs!**

MMIO: I/O Share Part of the Data Bus Address



The space of **32** bits address bus is **4GB**(0x0000_0000~0xFFFF_FFFF)

1024 bytes(0xFFFF_FC00~0xFFFF_FFFF) is designed to be allocated for the **I/O**.
Chip **S**elect and **address** are specified by specifying **10** IO port lines.

Here is an example for **24** LED lights and **24** DIP switches on Minisys board, both of them are divided into two groups, all the ports in one group share the same address.

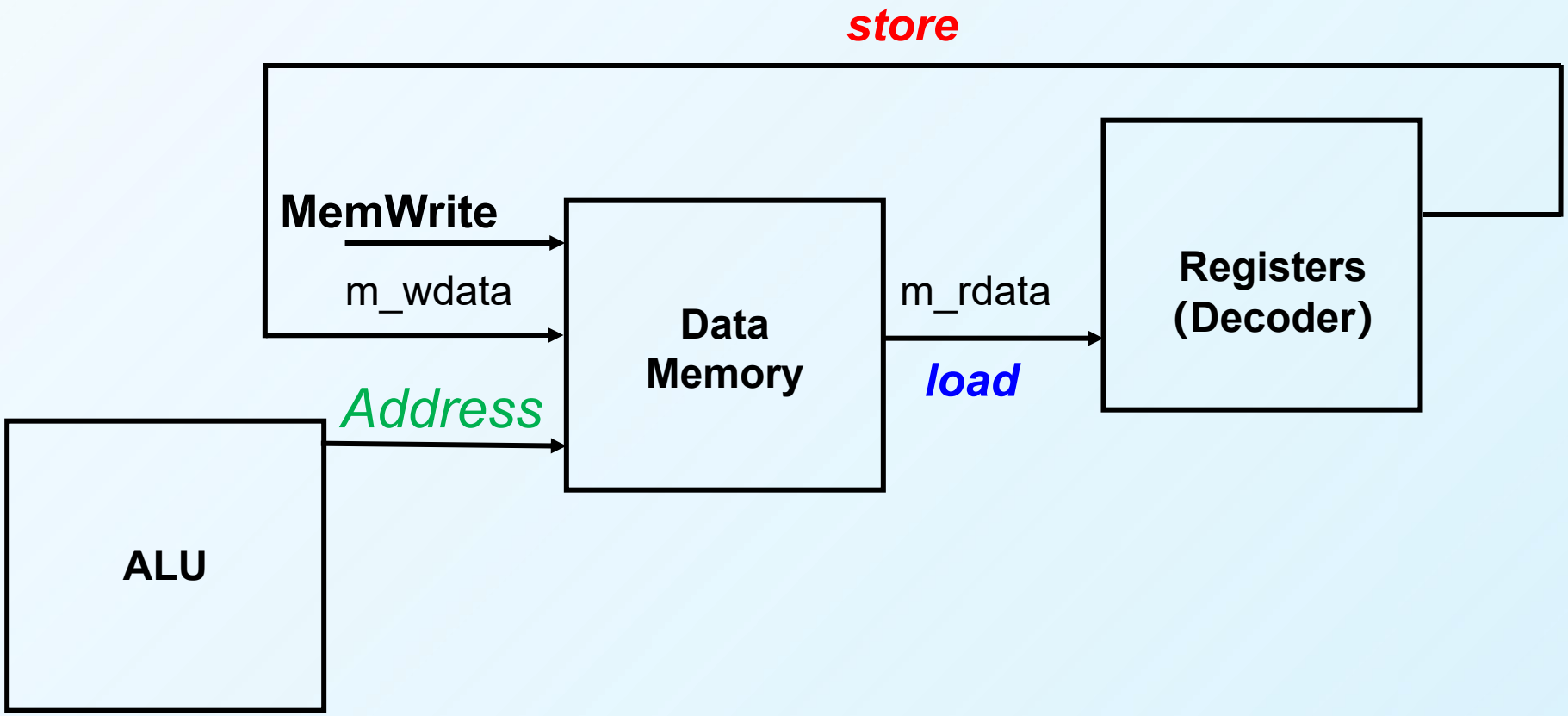
1. The CS(Chip Select) signal of the LED light is **ledCtrl**
2. The CS(Chip Select) signal of the DIP switch is **switchCtrl**

Range	LED(1~16)	LED(17~24)	Switch(1~16)	Switch(17~24)
Address	0x**FFFFFC60**	0x**FFFFFC62**	0x**FFFFFC70**	0x**FFFFFC72**

Note:
*1. In the computer field, there are usually two schemes for I/O address space design: I/O and memory **unified addressing** or **I/O independent addressing**. However there is no dedicated I/ O instruction in current Minisys-1. Here, both LW and SW instructions are used for RAM access and I/O access, which means Minisys-1 can only use I/O unified addressing.*

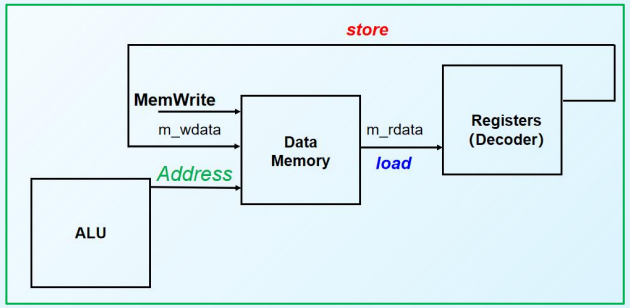
2. It is just a way for IO address implementation（MMIO: Memory-Mapped Input Output）, but not the only choice.

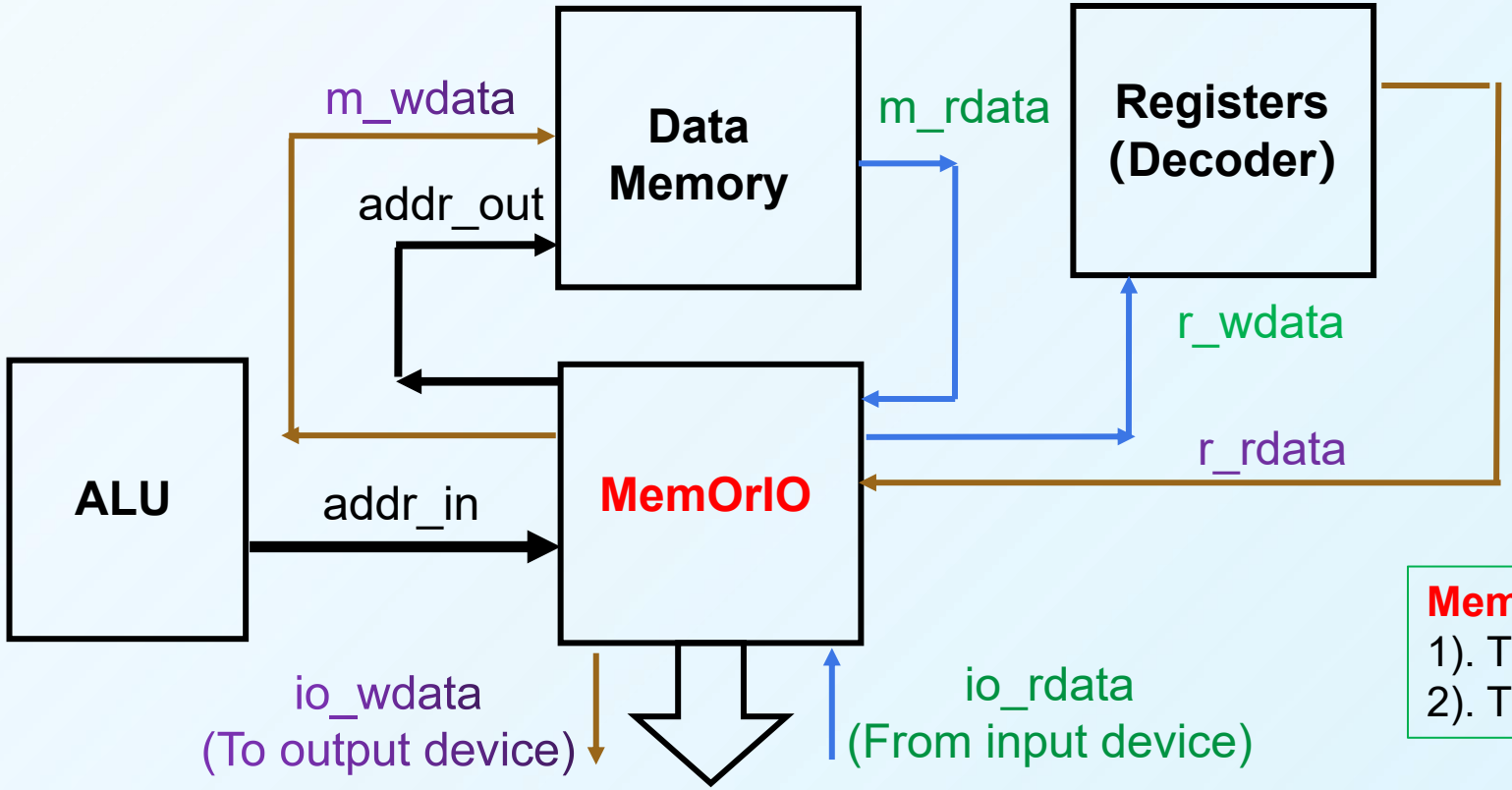
Corresponding Operation of LW/SW



NOTE:
1) There is no specific instruction in **MiniRISC-VI(lw, sw, add, sub, and, or, beq)** to read data from input ports and write data to output ports.
2) To implement the read/write process on I/O, it needs to **share the lw/sw instructions** in **MiniRISC-VI**.

MemOrIO





Chip Select signal

LedCtrl (0XFFFFFC60, 0XFFFFFC62)

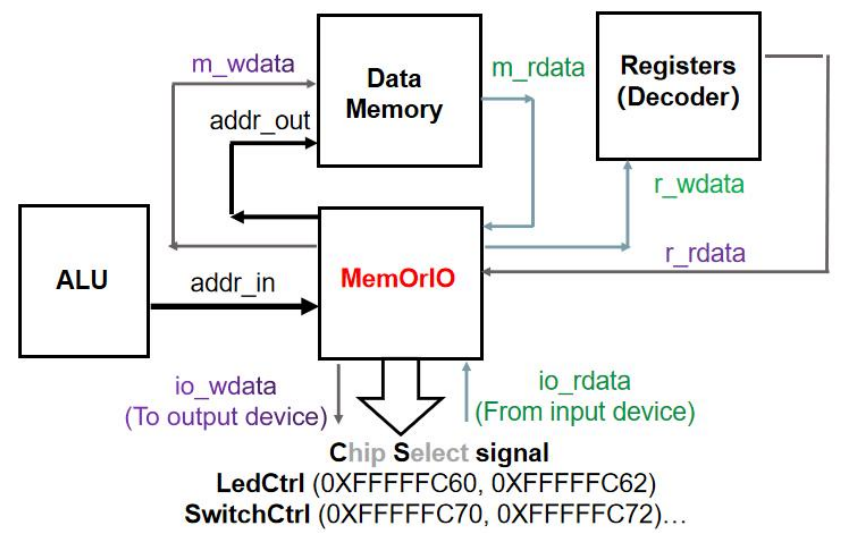
SwitchCtrl (0XFFFFFC70, 0XFFFFFC72)…

MemOrIO determine:

1). The source of r_wdata

2). The destination of r_rdata

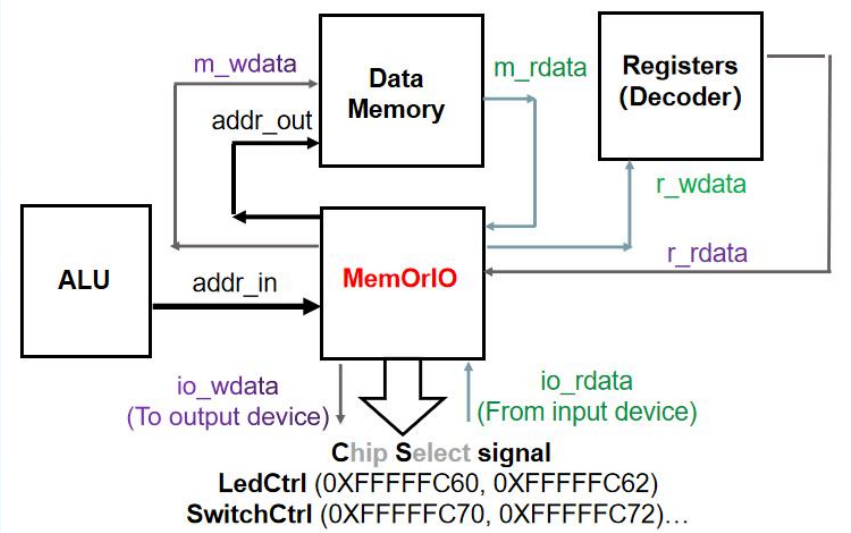
MemOrIO continued




```
module MemOrIO( mRead, mWrite, ioRead, ioWrite,addr_in, addr_out,
m_rdata, io_rdata, r_wdata, r_rdata, write_data, LEDCtrl, SwitchCtrl);

input mRead;                    // read memory, from Controller
input mWrite;                   // write memory, from Controller
input ioRead;                   // read IO, from Controller
input ioWrite;                  // write IO, from Controller

input[31:0] addr_in;             // from alu_result in ALU
output[31:0] addr_out;          // address to Data-Memory

input[31:0] m_rdata;            // data read from Data-Memory
input[15:0] io_rdata;           // data read from IO,16 bits
output[31:0] r_wdata;           // data to Decoder(register file)

input[31:0] r_rdata;        // data read from Decoder(register file)
output reg[31:0] write_data;    // data to memory or I/O  (m_wdata, io_wdata)
output LEDCtrl;                 // LED Chip Select
output SwitchCtrl;              // Switch Chip Select
```

*Tips: A demo about how the **Chip Select** signals work on **I/O** could be found in **labs/lab11_io** on course BlackBoard site*

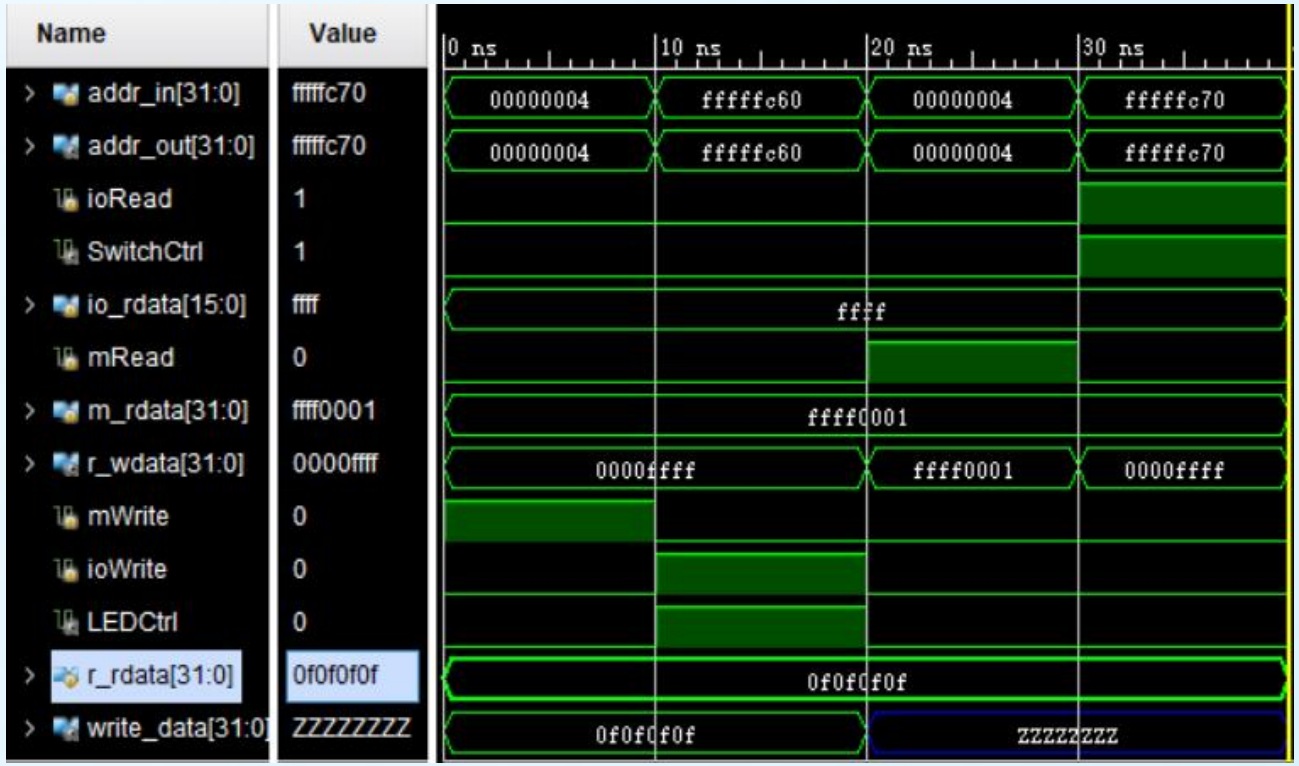
MemOrIO continued




```
assign addr_out= addr_in;
// The data wirte to register file may be from memory or io.
// While the data is from io, it should be the lower 16bit of r_wdata.
assign r_wdata = ?  ?  ?

// Chip select signal of  Led and Switch  are all active high;
assign LEDCtrl=  ?  ?  ?
assign SwitchCtrl= ?  ?  ?

always @* begin
        if((mWrite==1)||(ioWrite==1))
            //wirte_data could go to either memory or IO. where is it from?
                write_data = ?  ?  ?
        else
                write_data = 32'hZZZZZZZZ;
end
endmodule
```

The Function Verification of MemOrIO

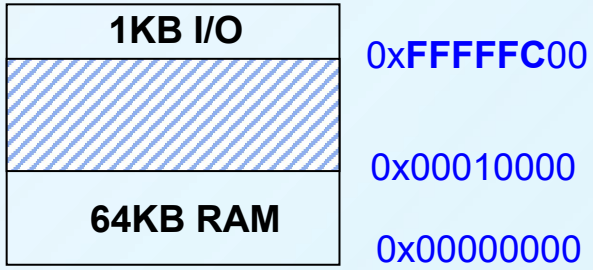
```
// a reference for the testbench of MemOrIO
module MemOrIO_tb( );
   reg mRead,mWrite,ioRead,ioWrite;
   reg[31:0] addr_in,m_rdata,r_rdata;
   reg[15:0] io_rdata;
   wire LEDCtrl,SwitchCtrl;
   wire [31:0] addr_out,r_wdata,write_data;

   MemoryOrIO umio(addr_out, addr_in,
mRead, mWrite, ioRead, ioWrite,
m_rdata, io_rdata, r_rdata, r_wdata, write_data,
LEDCtrl, SwitchCtrl );
```




```
 initial begin    // r_rdata -> m_wdata(write_data)
 m_rdata = 32'h0xffff_0001;  io_rdata = 16'h0xffff;  r_rdata = 32'h0x0f0f_0f0f;   addr_in = 32'h4;{mRead,mWrite,ioRead,ioWrite}= 4'b01_00;
 #10   addr_in = 32'hffff_fc60;   {mRead,mWrite,ioRead,ioWrite}= 4'b00_01;          // r_rdata -> io_wdata(write_data)
 #10  addr_in = 32'h0000_0004;   {mRead,mWrite,ioRead,ioWrite}= 4'b10_00;      // m_rdata -> r_wdata
 #10  addr_in = 32'hffff_fc70;    {mRead,mWrite,ioRead,ioWrite}= 4'b00_10;         // io_rdata -> r_wdata(write_data)
 #10 $finish;
  end
endmodule
```

Controller+

Add new ports to Controller for IO reading and writing support.




```
module  control32(Opcode,Function_opcode,Jr,Branch,nBranch,Jmp,Jal,
Alu_resultHigh,
RegDST, MemorIOtoReg, RegWrite,
MemRead, MemWrite,
IORead, IOWrite,
ALUSrc,ALUOp,Sftmd,I_format);
        ...
        input[21:0] Alu_resultHigh;     // From the execution unit Alu_Result[31..10]
        output MemorIOtoReg;    // 1 indicates that data needs to be read from memory or I/O to the register
        output RegWrite;        // 1 indicates that the instruction needs to write to the register
        output MemRead;         // 1 indicates that the instruction needs to read from the memory
        output MemWrite;        // 1 indicates that the instruction needs to write to the memory
        output IORead;          // 1 indicates  I/O read
        output IOWrite;         // 1 indicates  I/O write
        ...
```

Controller+ continued

1) **Modify** the logic of the '***MemWrite***'
2) **Add '*MemRead*'**, '***IORead***' and '***IOWrite***' signals
3) **Change '*MemtoReg*'** to '***MemorIOtoReg***'.

```
 // The real address of LW and SW is Alu_Result, the signal comes from the execution unit
// From the execution unit Alu_Result[31..10], used to help determine whether to process Mem or IO
  input[21:0]  Alu_resultHigh;

   output      MemorIOtoReg;        //1 indicates that read date from memory or I/O to write to the register
   output      MemRead;             // 1 indicates that reading from the memory to get data
   output      IORead;              // 1 indicates  I/O read
   output      IOWrite;             // 1 indicates  I/O write

   assign RegWrite = ? ? ?        // Write memory or write IO
   assign MemWrite = ((sw==1) && (Alu_resultHigh[21:0] != 22'h3FFFFF)) ? 1'b1:1'b0;
   assign MemRead = ?  ?  ?                  // Read memory
   assign IORead  = ?  ?  ?                  // Read input port
   assign IOWrite     = ?  ?  ?              // Write output port

// Read operations require reading data from memory or I/O to write to the register
   assign MemorIOtoReg = IORead || MemRead;
```

Practice

P1. Complete the following modules, do the function verification:

- 1. MemoryOrIO
- 2. Controller+
- 3. Single cycle CPU with I/O process

P2. Redesign and implement the solution about I/O data bus and I/O addressing that are suitable for your design. Build the single cycle CPU with the updated solution of I/O process and do the function verification.